



Sorting algorithms:

Let A be a list of n elements $A_1, A_2, \dots, A_n$ in memory. *Sorting* A refers

$$A_1 \leq A_2 \leq A_3 \leq \dots \leq A_n$$

Algorithm	Worst Case	Average Case
Bubble Sort	$\frac{n(n-1)}{2} = O(n^2)$	$\frac{n(n-1)}{2} = O(n^2)$
Quicksort	$\frac{n(n+3)}{2} = O(n^2)$	$1.4n \log n = O(n \log n)$
Heapsort	$3n \log n = O(n \log n)$	$3n \log n = O(n \log n)$



Sorting algorithms:

INSERTION SORT

Suppose an array A with n elements $A[1], A[2], \dots, A[N]$ is in memory.

Pass 1. $A[1]$ by itself is trivially sorted.

Pass 2. $A[2]$ is inserted either before or after $A[1]$ so that: $A[1], A[2]$ is sorted.

Pass 3. $A[3]$ is inserted into its proper place in $A[1], A[2]$, that is, before $A[1]$, between $A[1]$ and $A[2]$, or after $A[2]$, so that: $A[1], A[2], A[3]$ is sorted.

Pass 4. $A[4]$ is inserted into its proper place in $A[1], A[2], A[3]$ so that:

$A[1], A[2], A[3], A[4]$ is sorted.

.....
Pass N . $A[N]$ is inserted into its proper place in $A[1], A[2], \dots, A[N-1]$ so that:

$A[1], A[2], \dots, A[N]$ is sorted.

Sorted after Pass 1

5	3	7	10	0	11	8
---	---	---	----	---	----	---

Next
Item



Sorting algorithms:

INSERTION SORT

Suppose an array A with n elements $A[1], A[2], \dots, A[N]$ is in memory.

Pass 1. $A[1]$ by itself is trivially sorted.

Pass 2. $A[2]$ is inserted either before or after $A[1]$ so that: $A[1], A[2]$ is sorted.

Pass 3. $A[3]$ is inserted into its proper place in $A[1], A[2]$, that is, before $A[1]$, between $A[1]$ and $A[2]$, or after $A[2]$, so that: $A[1], A[2], A[3]$ is sorted.

Pass 4. $A[4]$ is inserted into its proper place in $A[1], A[2], A[3]$ so that:

$A[1], A[2], A[3], A[4]$ is sorted.

.....
Pass N . $A[N]$ is inserted into its proper place in $A[1], A[2], \dots, A[N-1]$ so that:

$A[1], A[2], \dots, A[N]$ is sorted.

Sorted after Pass 2

3	5	7	10	0	11	8
---	---	---	----	---	----	---

Next
Item



Sorting algorithms:

INSERTION SORT

Suppose an array A with n elements $A[1], A[2], \dots, A[N]$ is in memory.

Pass 1. $A[1]$ by itself is trivially sorted.

Pass 2. $A[2]$ is inserted either before or after $A[1]$ so that: $A[1], A[2]$ is sorted.

Pass 3. $A[3]$ is inserted into its proper place in $A[1], A[2]$, that is, before $A[1]$, between $A[1]$ and $A[2]$, or after $A[2]$, so that: $A[1], A[2], A[3]$ is sorted.

Pass 4. $A[4]$ is inserted into its proper place in $A[1], A[2], A[3]$ so that:

$A[1], A[2], A[3], A[4]$ is sorted.

.....
Pass N . $A[N]$ is inserted into its proper place in $A[1], A[2], \dots, A[N-1]$ so that:

$A[1], A[2], \dots, A[N]$ is sorted.

Sorted after Pass 3

3	5	7	10	0	11	8
---	---	---	----	---	----	---

Next
Item



Sorting algorithms:

INSERTION SORT

Suppose an array A with n elements $A[1], A[2], \dots, A[N]$ is in memory.

Pass 1. $A[1]$ by itself is trivially sorted.

Pass 2. $A[2]$ is inserted either before or after $A[1]$ so that: $A[1], A[2]$ is sorted.

Pass 3. $A[3]$ is inserted into its proper place in $A[1], A[2]$, that is, before $A[1]$, between $A[1]$ and $A[2]$, or after $A[2]$, so that: $A[1], A[2], A[3]$ is sorted.

Pass 4. $A[4]$ is inserted into its proper place in $A[1], A[2], A[3]$ so that:
 $A[1], A[2], A[3], A[4]$ is sorted.

.....
Pass N . $A[N]$ is inserted into its proper place in $A[1], A[2], \dots, A[N - 1]$ that:
 $A[1], A[2], \dots, A[N]$ is sorted.

Sorted after Pass 4

3	5	7	10	0	11	8
---	---	---	----	---	----	---

Next
Item



Sorting algorithms:

INSERTION SORT

Suppose an array A with n elements $A[1], A[2], \dots, A[N]$ is in memory.

Pass 1. $A[1]$ by itself is trivially sorted.

Pass 2. $A[2]$ is inserted either before or after $A[1]$ so that: $A[1], A[2]$ is sorted.

Pass 3. $A[3]$ is inserted into its proper place in $A[1], A[2]$, that is, before $A[1]$, between $A[1]$ and $A[2]$, or after $A[2]$, so that: $A[1], A[2], A[3]$ is sorted.

Pass 4. $A[4]$ is inserted into its proper place in $A[1], A[2], A[3]$ so that:
 $A[1], A[2], A[3], A[4]$ is sorted.

.....
Pass N . $A[N]$ is inserted into its proper place in $A[1], A[2], \dots, A[N - 1]$ so that:
 $A[1], A[2], \dots, A[N]$ is sorted.

Sorted after Pass 5

0	3	5	7	10	11	8
---	---	---	---	----	----	---

Next
Item



Sorting algorithms:

INSERTION SORT

Suppose an array A with n elements $A[1], A[2], \dots, A[N]$ is in memory.

Pass 1. $A[1]$ by itself is trivially sorted.

Pass 2. $A[2]$ is inserted either before or after $A[1]$ so that: $A[1], A[2]$ is sorted.

Pass 3. $A[3]$ is inserted into its proper place in $A[1], A[2]$, that is, before $A[1]$, between $A[1]$ and $A[2]$, or after $A[2]$, so that: $A[1], A[2], A[3]$ is sorted.

Pass 4. $A[4]$ is inserted into its proper place in $A[1], A[2], A[3]$ so that:
 $A[1], A[2], A[3], A[4]$ is sorted.

.....
Pass N . $A[N]$ is inserted into its proper place in $A[1], A[2], \dots, A[N - 1]$ so that
 $A[1], A[2], \dots, A[N]$ is sorted.

Sorted after Pass 6

0	3	5	7	10	11	8
---	---	---	---	----	----	---

Next
Item



Sorting algorithms:

INSERTION SORT

Suppose an array A with n elements $A[1], A[2], \dots, A[N]$ is in memory.

Pass 1. $A[1]$ by itself is trivially sorted.

Pass 2. $A[2]$ is inserted either before or after $A[1]$ so that: $A[1], A[2]$ is sorted.

Pass 3. $A[3]$ is inserted into its proper place in $A[1], A[2]$, that is, before $A[1]$, between $A[1]$ and $A[2]$, or after $A[2]$, so that: $A[1], A[2], A[3]$ is sorted.

Pass 4. $A[4]$ is inserted into its proper place in $A[1], A[2], A[3]$ so that:

$A[1], A[2], A[3], A[4]$ is sorted.

.....
Pass N . $A[N]$ is inserted into its proper place in $A[1], A[2], \dots, A[N - 1]$ so that:

$A[1], A[2], \dots, A[N]$ is sorted.

Sorted after Pass 7

0	3	5	7	8	10	11
---	---	---	---	---	----	----



Sorting algorithms:

INSERTION SORT

List of variables:

A: Integer
TEMP: Integer
PTR: Integer
K: Integer
N: Integer

List of loops:

For loop in Step 2
While loop in step 4

(Insertion Sort) INSERTION(A, N).
This algorithm sorts the array A with N elements.

1. Set $A[0] := -\infty$. [Initializes sentinel element.]
2. Repeat Steps 3 to 5 for $K = 2, 3, \dots, N$.
3. Set $TEMP := A[K]$ and $PTR := K - 1$.
4. Repeat while $TEMP < A[PTR]$.
 - (a) Set $A[PTR + 1] := A[PTR]$. [Moves element forward.]
 - (b) Set $PTR := PTR - 1$.
5. [End of loop.]
Set $A[PTR + 1] := TEMP$. [Inserts element in proper place.]
[End of Step 2 loop.]
6. Return.



CSE 203: Data Structure

Data
Structure

Sorting algorithms:

INSERTION SORT

List of variables:

A: Integer
TEMP: Integer
PTR: Integer
K: Integer

List of loops:

For loop in Step 2
While loop in step 4

```
main(){  
int A[10], ITEM, PTR, K, N=9;
```

(Insertion Sort) INSERTION(A, N).
This algorithm sorts the array A with N elements

1. Set $A[0] := -\infty$. [Initializes sentinel element]
2. Repeat Steps 3 to 5 for $K = 2, 3, \dots, N$:
3. Set $TEMP := A[K]$ and $PTR := K - 1$.
4. Repeat while $TEMP < A[PTR]$:
 - (a) Set $A[PTR + 1] := A[PTR]$.
 - (b) Set $PTR := PTR - 1$.[End of loop.]
5. Set $A[PTR + 1] := TEMP$. [Inserts element]
[End of Step 2 loop.]
6. Return.



CSE 203: Data Structure

Data
Structure

Sorting algorithms:

INSERTION SORT

List of variables:

A: Integer
TEMP: Integer
PTR: Integer
K: Integer

List of loops:

For loop in Step 2
While loop in step 4

```
main(){  
int A[10], ITEM, PTR, K, N=9;  
A[0]=-99999;  
for(K=2;K<=N;K++)           //Step 2  
{
```

(Insertion Sort) INSERTION(A, N).
This algorithm sorts the array A with N elements

1. Set $A[0] := -\infty$. [Initializes sentinel element]
2. Repeat Steps 3 to 5 for $K = 2, 3, \dots, N$:
3. Set $TEMP := A[K]$ and $PTR := K - 1$.
4. Repeat while $TEMP < A[PTR]$:
 - (a) Set $A[PTR + 1] := A[PTR]$.
 - (b) Set $PTR := PTR - 1$.[End of loop.]
5. Set $A[PTR + 1] := TEMP$. [Inserts element]
[End of Step 2 loop.]
6. Return.



CSE 203: Data Structure

Data
Structure

Sorting algorithms:

INSERTION SORT

List of variables:

A: Integer
TEMP: Integer
PTR: Integer
K: Integer

List of loops:

For loop in Step 2
While loop in step 4

```
main(){  
int A[10], ITEM, PTR, K, N=9;  
A[0]=-99999;  
for(K=2;K<=N;K++)           //Step 2  
{  
    TEMP=A[K];                //Step 3  
    PTR=K-1                    //Step3  
    while (A[PTR]>ITEM)        //Step 4  
    {  
        PTR=PTR-1;  
    }  
    A[PTR+1]=ITEM;             //Step 5  
}
```

(Insertion Sort) INSERTION(A, N).
This algorithm sorts the array A with N elements

1. Set $A[0] := -\infty$. [Initializes sentinel element]
2. Repeat Steps 3 to 5 for $K = 2, 3, \dots, N$:
Set $TEMP := A[K]$ and $PTR := K - 1$.
3. Repeat while $TEMP < A[PTR]$:
(a) Set $A[PTR + 1] := A[PTR]$.
(b) Set $PTR := PTR - 1$.
4. [End of loop.]
Set $A[PTR + 1] := TEMP$. [Inserts element]
5. [End of Step 2 loop.]
6. Return.



CSE 203: Data Structure

Data
Structure

Sorting algorithms:

INSERTION SORT

List of variables:

A: Integer
TEMP: Integer
PTR: Integer
K: Integer

List of loops:

For loop in Step 2
While loop in step 4

```
main(){
int A[10], ITEM, PTR, K, N=9;
A[0]=-99999;
for(K=2;K<=N;K++)
{
TEMP=A[K];
PTR=K-1
while(TEMP<A[PTR])           //Step 4
{
A[PTR+1]=A[PTR];
PTR=PTR-1;
}
```

(Insertion Sort) INSERTION(A, N).
This algorithm sorts the array A with N elements

1. Set $A[0] := -\infty$. [Initializes sentinel element]
2. Repeat Steps 3 to 5 for $K = 2, 3, \dots, N$:
3. Set $TEMP := A[K]$ and $PTR := K - 1$.
4. Repeat while $TEMP < A[PTR]$:
 - (a) Set $A[PTR + 1] := A[PTR]$.
 - (b) Set $PTR := PTR - 1$.[End of loop.]
5. Set $A[PTR + 1] := TEMP$. [Inserts element]
6. Return.

[End of Step 2 loop.]



CSE 203: Data Structure

Data
Structure

Sorting algorithms:

INSERTION SORT

List of variables:

A: Integer
TEMP: Integer
PTR: Integer
K: Integer

List of loops:

For loop in Step 2
While loop in step 4

```
main(){
int A[10], ITEM, PTR, K, N=9;
A[0]=-99999;
for(K=2;K<=N;K++)
{
TEMP=A[K];
PTR=K-1
while(TEMP<A[PTR])
{
A[PTR+1]=A[PTR];
PTR=PTR-1;
}
A[PTR+1]=TEMP;           //Step 5
}
}                          //Step 6
```

(Insertion Sort) INSERTION(A, N).
This algorithm sorts the array A with N elements

1. Set $A[0] := -\infty$. [Initializes sentinel element]
2. Repeat Steps 3 to 5 for $K = 2, 3, \dots, N$:
Set $TEMP := A[K]$ and $PTR := K - 1$.
3. Repeat while $TEMP < A[PTR]$:
(a) Set $A[PTR + 1] := A[PTR]$.
(b) Set $PTR := PTR - 1$.
[End of loop.]
5. Set $A[PTR + 1] := TEMP$. [Inserts element]
[End of Step 2 loop.]
6. Return.